

Supplemental

Eric D. Hermes, Aurora N. Janes, and J.R. Schmidt*

Department of Chemistry and Theoretical Chemistry Institute, University of Wisconsin–Madison, 1101 University Ave., Madison, WI 53706, United States

E-mail: schmidt@chem.wisc.edu

S1. Derivation of lattice partition function

Given a lattice of M sites containing N_i many of each adsorbate i , each of which has a one-particle partition function q_i , and assuming each adsorbate occupies only a single site, the full partition function for the lattice is

$$Q_{\text{latt}}(\{N_i\}) = \prod_i \prod_{n=1}^{N_i} \frac{\left(M - \sum_{j < i} N_j - (n-1)\right)}{n} q_i \quad (\text{S1})$$

$$= \frac{M!}{(M - \sum_i N_i)!} \prod_i \frac{q_i^{N_i}}{N_i!} \quad (\text{S2})$$

$$= \frac{M!}{N_*!} \prod_i \frac{q_i^{N_i}}{N_i!}, \quad (\text{S3})$$

where we have defined $N_* \equiv M - \sum_i s_i N_i$ to be the number of vacant sites on the lattice, where s_i is equal to the number of sites occupied by species i .

S2. Derivation of Idealized PFR Steady State Algebraic Differential Equations

The mass balance equation for a PFR is given by

$$\frac{\partial([C_i]A)}{\partial t} = -Q \frac{\partial[C_i]}{\partial z} + f(\{[C], \theta_C\})A \quad (\text{S4})$$

$$\frac{\partial[C_i]}{\partial t} = -\frac{Q}{A} \frac{\partial[C_i]}{\partial z} + f(\{[C], \theta_C\}) \quad (\text{S5})$$

where Q is the flow rate, A is the cross sectional area of the reactor column and $f(\{[C], \theta_C\})$ represents the rates of reaction processes that form and consume fluid-phase species C_i . The first term on the right hand side represents the flow of C_i out of a region of the reactor, while the second term represents the generation / consumption of the species due to reactions.

At steady state, the left hand side of the equation is zero and therefore

$$\frac{Q}{A} \frac{d[C_i]}{dz} = f(\{[C], \theta_C\}). \quad (\text{S6})$$

Changing variables, dz can be transformed into the space-time $d\tau$,

$$d\tau = \frac{dV}{Q} = \frac{Adz}{Q}, \quad (\text{S7})$$

yielding

$$\frac{d[C]}{d\tau} = f(\{[C], \theta_C\}). \quad (\text{S8})$$

For a surface species, θ_{C_i} , the above derivation is the same, however since surface adsorbed species are not subject to flow equation S6 simplifies to

$$0 = g(\{[C], \theta_C\}) \quad (\text{S9})$$

where $g(\{[C], \theta_C\})$ represent the possible rates that form and consume surface species C .

TOFs at all 21 Experimental Conditions

Table S1: Experimental conditions and TOF for experiment, Grabow et. al. model and our model for all 21 conditions. Condition 7 is the condition referenced in the paper. Temperature is in units of Kelvin, flow rate is in cm³/min and TOF are in units of [min⁻¹] and

Id #	Temp	y(CO)	y(H ₂ O)	y(CO ₂)	y(H ₂)	Flowrate	exp TOF	Grabow TOF	Micki TOF
1	523	0.154	0.208	0	0	102.9	3.68	3.38	3.44
2	548	0.055	0.208	0	0	85.9	8.56	10.01	7.98
3	548	0.105	0.208	0	0	97.4	8.06	7.51	7.37
4	548	0.137	0.062	0	0	118	3.63	2.93	2.89
5	548	0.144	0.104	0	0	114.8	4.81	4.36	4.28
6	548	0.148	0.145	0	0	108.4	5.56	5.31	5.43
7	548	0.145	0.208	0	0	101.4	7.27	6.87	7.06
8	548	0.106	0.208	0.068	0	106.9	6.05	7.65	7.55
9	548	0.104	0.208	0.109	0	105.6	5.59	7.63	7.55
10	548	0.14	0.208	0.151	0	110.1	6.12	7.01	7.27
11	548	0.102	0.208	0.192	0	109.1	6.03	7.67	7.64
12	548	0.134	0.208	0	0.037	103.5	4.13	4.94	4.20
13	548	0.156	0.208	0	0.037	102.1	2.77	3.21	4.08
14	548	0.13	0.208	0	0.123	105.9	2.67	3.1	2.72
15	548	0.134	0.208	0.177	0.123	95.7	2.67	2.95	2.69
16	548	0.132	0.208	0	0.173	103.8	2.55	2.91	2.35
17	548	0.145	0.208	0	0.191	94.4	2.28	2.38	2.22
18	548	0.159	0.208	0	0.208	101.1	2.29	2.54	2.11
19	548	0.198	0.208	0	0	102.6	7.29	6.31	6.74
20	548	0.223	0.208	0	0	88.3	7.09	5.76	6.34
21	573	0.15	0.208	0	0	103.4	15.44	13.2	13.16

Code Snippets

Listing S1: An example script illustrating how to calculate Campbell's degree of rate control. This method is included in the module `analysis.py`. We use adsorption of CO as a proxy for the production of CO₂. In our reaction network, all net adsorption of CO must necessarily result in formation of CO₂. Due to the presence of multiple CO₂-producing elementary steps, it is easier to measure the net production of CO₂ by inspecting the net adsorption of CO.

```
1      chi_rc = {}
2      rmid = r1['co_ads'] # as a proxy for CO2 production
3      scale = 0.001
4      for rname, rxn in rxns.items():
5          rxn.set_scale('kfor', 1 - scale)
6          rxn.set_scale('krev', 1 - scale)
7          model.set_initial_conditions(U1)
8          t2, U2, r2 = model.find_steady_state()
9          rlow = r2['co_ads']
10
11         rxn.set_scale('kfor', 1 + scale)
12         rxn.set_scale('krev', 1 + scale)
13         model.set_initial_conditions(U1)
14         t3, U3, r3 = model.find_steady_state()
15         rhigh = r3['co_ads']
16
17         rxn.set_scale('kfor', 1)
18         rxn.set_scale('krev', 1)
19
20     chi_rc[rname] = (rhigh - rlow) / (2 * scale * rmid)
```

Listing S2: An example script illustrating how to calculate Campbell's degree of thermodynamic rate control. This method is included in the module `analysis.py`.

```

1      from ase.units import kB
2      from micki import Adsorbate
3
4      chi_trc = {}
5      rmid = r1['co_ads'] # as a proxy for CO2 production
6      scale = 0.001
7      dE = scale * kB * T
8      for name, species in sp.items()
9          # only calculate chi_trc for adsorbed minima
10         if (not isinstance(species, Adsorbate)
11             or species.ts
12             or name == 'slab'):
13             continue
14
15         species.dE -= dE
16         for _, rxn in rxns.items():
17             rxn.update(T=T, Asite=Asite, force=True)
18             model.set_initial_conditions(U1)
19             t2, U2, r2 = model.find_steady_state()
20             rlow = r2['co_ads']
21
22         species.dE += 2*dE
23         for _, rxn in rxns.items():
24             rxn.update(T=T, Asite=Asite, force=True)
25             model.set_initial_conditions(U1)
26             t3, U3, r3 = model.find_steady_state()
27             rhigh = r3['co_ads']
28
29         species.dE -= dE
30
31         chi_trc[name] = (rlow - rhigh) / (rmid * scale)

```

Listing S3: An example script illustrating how to calculate the rate order of reactants. This method is included in the module `analysis.py`.

```

1     rate_order = {}
2     rmid = r1['co_ads'] # as a proxy for CO2 production
3     drho = 5e-5
4     for name in ['co_g', 'h2o_g']:
5         species = sp[name]
6         rhomid = U0[name]
7
8     U0_new = U0.copy()
9     U0_new[name] -= drho
10    model.set_initial_conditions(U0_new)
11    t2, U2, r2 = model.find_steady_state()
12    rlow = r2['co_ads']
13
14    U0_new[name] += 2*drho
15    model.set_initial_conditions(U0_new)
16    t3, U3, r3 = model.find_steady_state()
17    rhigh = r3['co_ads']
18
19    rate_order[name] = rhomid * (rhigh - rlow) / (2 * drho * rmid)

```

Listing S4: An example script illustrating how to calculate the effective activation energy of a model. This method is included in the module `analysis.py`.

```

1     from ase.units import kB
2
3     rmid = r1['co_ads'] # as a proxy for CO2 production
4     dT = 0.01
5
6     model.T = T - dT
7     model.set_initial_conditions(U1)
8     t2, U2, r2 = model.find_steady_state()
9     rlow = r2['co_ads']
10
11    model.T = T + dT
12    model.set_initial_conditions(U1)
13    t3, U3, r3 = model.find_steady_state()
14    rhigh = r3['co_ads']
15
16    model.T = T
17    Ea = kB * T**2 * (rhigh - rlow) / (2 * rmid * dT)

```